Supplementary Methods and Reproducibility Evidence

Ali Al Bataineh
Computer Information Science, Higher Colleges of Technology
Abu Dhabi, United Arab Emirates
aalbataineh@hct.ac.ae

This supplement accompanies *Checking Machine-Learning Workflows Through Evidence Validation, Python Replay, and Counterfactual Tests*. The software release is TraceSplit 0.1.0. Every number below is generated from the saved result files by the script `scripts/fill_manuscript.py` in the artifact.

1 Exact program grammar

The following is the distributed Lark grammar. Statement order, allowed names, parameter ranges, and role restrictions are checked after parsing. Each statement occupies one logical line.

```
start: (_NL | statement _NL)*
?statement: read | predict | drop | split | prepare | train | evaluate | select | assess |
    explain
read: "read" STRING
predict: "predict" STRING "as" NAME
drop: "drop" STRING+
split: "split" NAME ["by" STRING] "train" PERCENT "validation" PERCENT "test" PERCENT "seed"
    INT
prepare: "prepare" "numeric" NAME "scale" NAME "categorical" NAME "on" NAME
train: "train" NAME NAME parameter* "on" NAME
parameter: NAME "=" NUMBER
evaluate: "evaluate" NAME "on" NAME
select: "select" "best" "by" NAME
assess: "assess" "best" "on" NAME
explain: "explain"
PERCENT: /[0-9]+(\.[0-9]+)?%/
NUMBER: /[+-]?[0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?/
%import common.CNAME -> NAME
%import common.INT
%import common.ESCAPED_STRING -> STRING
%import common.NEWLINE -> _NL
%import common.WS_INLINE
%ignore WS_INLINE
%ignore /#[^\n]*/
```

2 Checker protocol and boundaries

The checker tracks an ordered phase and the sets of trained and evaluated candidates. A read advances to target declaration; optional drops precede the unique split; the unique preparation follows the split.

Training and validation evaluation can be interleaved. Selection requires a nonempty candidate set and equality of the trained and evaluated sets. Assessment follows selection and is terminal, except for one optional explanation.

The normalized plan retains the source text, its SHA-256 fingerprint, and the source lines. An immutable plan marker prevents accidental bypass through the ordinary API; it is not a security mechanism against callers able to import internal Python objects. The standard-library evidence checker validates the normalized protocol and line metadata but does not parse the source text a second time.

The evidence record contains zero-based positional row identifiers. Partition checks require exhaustive, disjoint coverage and at least two rows per partition. Grouped splits forbid a group appearing in two partitions. Temporal splits require strictly ordered boundary keys. Source-level rules exclude the target and split metadata from predictors. They cannot identify undeclared proxies, mislabeled entities, or leakage introduced before CSV loading.

3 Numerical configuration

All eight numerical programs use 60/20/20 declared proportions and seeds 0, 1, 7, 42, 2026. Numeric imputation is median; scaling is standard; categorical imputation is most-frequent with one-hot encoding. Logistic regression uses $C = 1$, the limited-memory Broyden-Fletcher-Goldfarb-Shanno (LBFGS) solver, and at most 2,000 iterations. Ridge regression uses $\alpha = 1$ and a Cholesky solver. Forests use 40 trees and depth 6. Decision trees use depth 5. Forest workers and numerical library threads are limited to one during execution. Candidate selection uses accuracy for classification and RMSE for regression. Ties retain declaration order. There is no refit on validation.

The stored validation and test metrics are accuracy, balanced accuracy, and macro-F1 for classification, and RMSE, MAE, and R^2 for regression. The checker recomputes these scalar metrics from saved predictions and CSV labels with its own arithmetic. The replay compares all candidate validation predictions and metrics as well as the selected candidate's test outputs, at absolute and relative tolerance 10^{-10}.

4 Constructed source cases

The 48 valid source cases combine eight program templates with three numeric imputation methods and two scaling settings. The 192 invalid cases apply 24 families to each template, eight programs per family. Table 1 lists the families. All 240 cases received their expected decision.

Table 1: Constructed invalid-source families and the modification applied to each template.

Family	Modification
arbitrary_python	A Python import statement appended to the program.
assess_validation	Final assessment on the validation partition.
bad_parameter_range	Forest size set to zero.
bad_percentages	Training proportion changed so that the proportions do not sum to 100.
bad_seed	Seed equal to 2^{32}.
drop_target	The target column dropped before the split.
duplicate_candidate	Two candidates with the same name.

Family	Modification
duplicate_parameter	The same parameter given twice.
evaluate_test	Validation evaluation directed at the test partition.
fit_validation	The first candidate fitted on the validation partition.
missing_assessment	Assessment statement removed.
post_assessment_fit	A training statement after assessment.
prepare_all	Preparation fitted on all rows.
preprocess_before_split	Preparation statement moved before the split.
repeated_prepare	Two preparation statements.
repeated_split	Two split statements.
second_assessment	A second assessment statement.
select_too_early	Selection before every candidate has been evaluated.
undefined_candidate	Evaluation of a candidate that was never trained.
unknown_parameter	A parameter the algorithm does not accept.
unknown_task	A task other than classification or regression.
unsupported_algorithm	An algorithm outside the supported set.
wrong_metric	A selection metric of the other task.
wrong_task_algorithm	A regression algorithm in a classification program, or the converse.

5 Altered evidence records

Each family in Table 2 was applied once to each of the 40 numerical workflows. Where applicable, the mutation generator recomputes chain hashes after changing a semantic field. 2 families remain well formed without data but become checkable when the source CSV is available.

Table 2: Record mutations and observed rejection counts with and without the CSV.

Family	Cases	With CSV	Without CSV
assess_scope	40	40	40
broken_chain	40	40	40
dataset_fingerprint	40	40	0
evaluate_scope	40	40	40
fit_scope	40	40	40
metric_recomputed_chain	40	40	0
model_fingerprint	40	40	40
partition_omission	40	40	40
partition_overlap	40	40	40
prepare_fit_scope	40	40	40
preprocessing_fingerprint	40	40	40
second_assessment	40	40	40
source_fingerprint	40	40	40
source_location	40	40	40
wrong_selection	40	40	40
Total	600	600	520

The dataset-fingerprint mutation cannot be rejected merely because the altered hash has a different value. Without the original bytes, the checker has no external value against which to compare it. Likewise, a test metric that is changed consistently throughout the record needs actual labels and recorded predictions to be recomputed.

6 Counterfactual construction

The feature intervention modifies only held-out predictor rows while preserving the fixed partition. It applies $x' = 11x + 17$ to numeric test values, preserving missing entries, and replaces categorical test values with `__HOLDOUT_ONLY_CATEGORY__`, within an accepted schema. The label intervention rotates classification labels through the existing class set or adds 17 to regression test labels. Training and validation values are unchanged. Source paths are rewritten only to point to the temporary CSV. Ordinary source programs do not expose the frozen-partition override.

The expected unchanged state consists of preprocessing statistics, fitted-model fingerprints, validation metric dictionaries, and the winner. Test scores and predictions may change and are not part of the invariant. There are 80 correct-engine comparisons, two interventions for each input and seed, and all 80 passed.

7 Injected-fault matrix

Table 3 gives the complete outcome counts for the 9 mechanisms. Table 4 gives, for each mechanism and input, the number of the five runs in which the echo record was rejected with the CSV, the replay disagreed, and the perturbation changed development state.

Table 3: Injected faults: cases, executions whose compared outputs differ from the correct run (Changed; undefined for the data-level fault), honest records rejected with the CSV (Honest), echo records rejected with and without the CSV (Echo, Echo$^-$), replay disagreements (Replay), and development-state changes under the held-out feature perturbation (Perturb.).

Mechanism	Cases	Changed	Honest	Echo	Echo$^-$	Replay	Perturb.
Preparation fitted on all rows	40	26	40	0	0	26	40
Preparation fitted on training and validation rows	40	23	40	0	0	23	0
Estimators fitted on training and validation rows	40	39	40	0	0	39	0
Test rows added to training rows	40	40	40	0	0	40	40
Selection by test score	40	10	40	10	10	10	19
Winner refitted on training and validation rows before assessment	40	27	40	0	0	27	0
Estimators fitted with an undeclared seed	40	33	0	0	0	33	0
Grouped or temporal rule ignored	10	10	0	10	0	0	0
Target proxy column in the data	40	n/a	0	0	0	0	0

Table 4: Injected faults by input. Each cell gives record/replay/perturbation detections over the five seeds, where record denotes rejection of the echo record with the CSV; a dash marks an inapplicable mechanism.

Mechanism	Iris	Wine	Breast c.	Digits	Diabetes	Mixed	Grouped	Temporal
Preparation fitted on all rows	0/2/5	0/3/5	0/4/5	0/3/5	0/5/5	0/4/5	0/0/5	0/5/5
Preparation fitted on training and validation rows	0/1/0	0/3/0	0/1/0	0/5/0	0/5/0	0/3/0	0/0/0	0/5/0
Estimators fitted on training and validation rows	0/4/0	0/5/0	0/5/0	0/5/0	0/5/0	0/5/0	0/5/0	0/5/0
Test rows added to training rows	0/5/5	0/5/5	0/5/5	0/5/5	0/5/5	0/5/5	0/5/5	0/5/5
Selection by test score	0/0/0	2/2/3	2/2/0	0/0/0	1/1/5	1/1/4	4/4/2	0/0/5
Winner refitted on training and validation rows before assessment	0/1/0	0/1/0	0/3/0	0/5/0	0/5/0	0/2/0	0/5/0	0/5/0
Estimators fitted with an undeclared seed	0/1/0	0/2/0	0/5/0	0/5/0	0/5/0	0/5/0	0/5/0	0/5/0
Grouped or temporal rule ignored	–	–	–	–	–	–	5/0/0	5/0/0
Target proxy column in the data	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0

The honest record of each mechanism is constructed as follows. For preparation on all rows or on training and validation rows, the preparation event names the rows actually used. For estimators fitted on training and validation rows or on training and test rows, each fit event names the rows actually used. For selection by the test score, the selection event names the test partition. For the refit before assessment, an additional fit event on the training and validation rows is inserted after selection. For the undeclared seed, the recorded plan carries the seed that was actually used. For the ignored partition rule, the recorded plan declares a random strategy and lists the metadata column among the dropped columns. The data-level mechanism has no dishonest variant; its honest and echo records coincide.

8 Matched comparison with mlw

Table 5 lists, for each family, the error code raised by the TraceSplit checker and the exception class raised by mlw 1.1.2, running on pandas 2.3.3 in a separate environment whose pinned versions are in `experiments/comparison/requirements-mlw.txt`. The partition registry of mlw was cleared before every case. After the run, its in-process audit chain held 184 entries and verified as intact. The mlw cases use the scikit-learn engine, the logistic or linear algorithm, an explicit task, and the default 60/20/20 ratio; grouped and temporal inputs pass the metadata column through the groups and time arguments of split.

Table 5: Error codes raised by each tool for each family on the eight inputs. A dash marks acceptance on every input.

Family	TraceSplit	mlw
Valid workflow	–	–
Fit before any split	E_ORDER	PartitionError
Preprocessing before the split	E_ORDER	–
Preparation fitted on all rows	E_FIT_SCOPE	–
Fit on validation rows	E_FIT_SCOPE	–
Fit on test rows	E_FIT_SCOPE	PartitionError
Test rows added to training rows	No statement can fit an estimator on rows outside the training partition.	PartitionError
Validation evaluation on test rows	E_EVAL_SCOPE	PartitionError
Selection by test score	Selection has no partition argument; it always uses validation scores.	PartitionError
Final assessment on validation rows	E_ASSESS_SCOPE	PartitionError
Second assessment of the same test rows	E_TERMINAL	PartitionError
Development operation after assessment	E_TERMINAL	–
Workflow without final assessment	E_INCOMPLETE	–

9 All observed selected-candidate test scores

Table 6 reports each run's selected candidate and descriptive test score. Candidate selection was based on validation, not on these scores. The table is included for traceability and supports no comparison between learning algorithms. The temporal split is deterministic, so the five temporal runs differ only in the forest seed, and the selected linear candidate yields the same score in each.

Table 6: Descriptive test outputs for all 40 numerical workflows.

Input	Seed	Metric	Selected	Test value
iris	0	accuracy	linear	0.96666667
iris	1	accuracy	linear	0.96666667
iris	7	accuracy	linear	0.93333333
iris	42	accuracy	linear	0.93333333
iris	2026	accuracy	linear	0.93333333
wine	0	accuracy	linear	0.97222222
wine	1	accuracy	linear	0.97222222
wine	7	accuracy	forest	1.00000000
wine	42	accuracy	linear	0.97222222
wine	2026	accuracy	linear	1.00000000
breast_cancer	0	accuracy	tree	0.93859649
breast_cancer	1	accuracy	linear	0.98245614
breast_cancer	7	accuracy	forest	0.94736842

Input	Seed	Metric	Selected	Test value
breast_cancer	42	accuracy	linear	0.99122807
breast_cancer	2026	accuracy	linear	0.99122807
digits	0	accuracy	linear	0.96388889
digits	1	accuracy	linear	0.95555556
digits	7	accuracy	linear	0.96388889
digits	42	accuracy	linear	0.96388889
digits	2026	accuracy	linear	0.97222222
diabetes	0	rmse	linear	55.42086945
diabetes	1	rmse	linear	52.42737201
diabetes	7	rmse	linear	51.93000381
diabetes	42	rmse	linear	57.08344544
diabetes	2026	rmse	linear	53.79377651
mixed	0	accuracy	linear	0.93125000
mixed	1	accuracy	linear	0.95000000
mixed	7	accuracy	linear	0.95625000
mixed	42	accuracy	linear	0.97500000
mixed	2026	accuracy	linear	0.93125000
grouped	0	accuracy	forest	0.55000000
grouped	1	accuracy	forest	0.71666667
grouped	7	accuracy	forest	0.68333333
grouped	42	accuracy	linear	0.77500000
grouped	2026	accuracy	forest	0.72500000
temporal	0	rmse	linear	1.13862574
temporal	1	rmse	linear	1.13862574
temporal	7	rmse	linear	1.13862574
temporal	42	rmse	linear	1.13862574
temporal	2026	rmse	linear	1.13862574

10 Timing and test evidence

The raw timing table has one row per paired observation: input rows, repetition, execution order, language time, precompiled-plan time, and their difference in milliseconds. The first 3 pairs at each size are warm-ups and are excluded from the 25 measured pairs. A seeded 10,000-resample percentile bootstrap is applied to the paired median. The observations include CSV loading and in-memory evidence creation but exclude interpreter startup, disk serialization, external validation, and the replay subprocess.

The test suite passed 296 tests with 0 failures. Statement coverage was 94% of 704 statements; the per-module values are in the machine-readable coverage report. A separate smoke script runs the command-line interface, the standard-library checker, the exported Python, and the numerical comparison in subprocesses. The checker can be invoked with `python -S`, which disables site-package initialization.

11 Artifact file map

Table 7 connects the implementation and recorded results to the reviewer tasks.

Table 7: Artifact files and their roles.

Path	Purpose
`src/tracesplit/grammar.lark`	Exact syntax.
`src/tracesplit/language.py`	Parser and source-level checking.
`src/tracesplit/engine.py`	Input validation, partition construction, preprocessing, and learning.
`src/tracesplit/verifier.py`	Standard-library record checking and metric recomputation.
`src/tracesplit/replay_template.py.txt`	Separately assembled numerical replay.
`experiments/reproduce.py`	Conformance, replay, altered-record, counterfactual, and runtime stages.
`experiments/fault_matrix.py`	Injected-fault mechanisms, honest and echo records, and the three oracles.
`experiments/comparison/`	Matched protocol-violation cases for both tools and the merge script.
`results/full/runs/`	All run records, predictions, explanations, replays, and exports.
`results/full/*_cases.csv`	Constructed cases and counterfactual outcomes.
`results/full/witness_mutations.csv`	Per-workflow altered-record outcomes.
`results/full/fault_matrix.csv`	Per-execution injected-fault outcomes.
`results/full/comparison/`	Per-input outcomes of the matched comparison.
`results/full/runtime_pairs.csv`	Raw measured runtime pairs.
`scripts/build_report.py`	Derived summaries and figures.
`scripts/fill_manuscript.py`	Numbers in the article and in this supplement.
`docs/REVIEWER_GUIDE.md`	Installation and expected outcomes.

12 Local interface

The local application is a standard-library HTTP interface with editable examples and tabs for results, the explanation, the evidence record, and the exported Python. It runs one bounded subprocess at a time, binds to the loopback address, and requires a per-run token. It is a convenience for inspection, not a hardened service. Its routes are covered by automated tests, and it was exercised end to end in a browser on the author's computer.